

Operating systems

Lecture #3

Content Lecture#3

01	I
02	P
03	C
04	O
05	P
06	P
07	P
08	P
09	P
10	Z

Lets take a Break



Software Engineer, noun

A person that is happy when children die before their parents do

Revising Previous Lecture

Important System Calls Used in OS

fork()
Processes use this system call to create processes that are a copy of themselves. With the help of this system Call parent process creates a child process, and the execution of the parent process will be suspended till the child process executes.

exec()
This system call runs when an executable file in the context of an already running process that replaces the older executable file. However, the original process identifier remains as a new process is not built, but stack, data, head, data, etc. are replaced by the new process.

exit()
The exit() system call is used to terminate program execution. Specially in the multi-threaded environment, this call defines that the thread execution is complete. The OS reclaims resources that were used by the process after the use of exit() system call.

kill():
The kill() system call is used by OS to send a termination signal to a process that urges the process to exit. However, a kill system call does not necessarily mean killing the process and can have various meanings.

wait()
In some systems, a process needs to wait for another process to complete its execution. This type of situation occurs when a parent process creates a child process, and the execution of the parent process remains suspended until its child process executes.
The suspension of the parent process automatically occurs with a wait() system call. When the child process ends execution, the control moves back to the parent process.



Scheduling

process scheduling, is the basis for multi programmed operating systems, Objective of maximum CPU utilization obtained with multiprogramming

Exec()

the program specified in the parameter to exec() will replace the entire process including thread

Scheduling

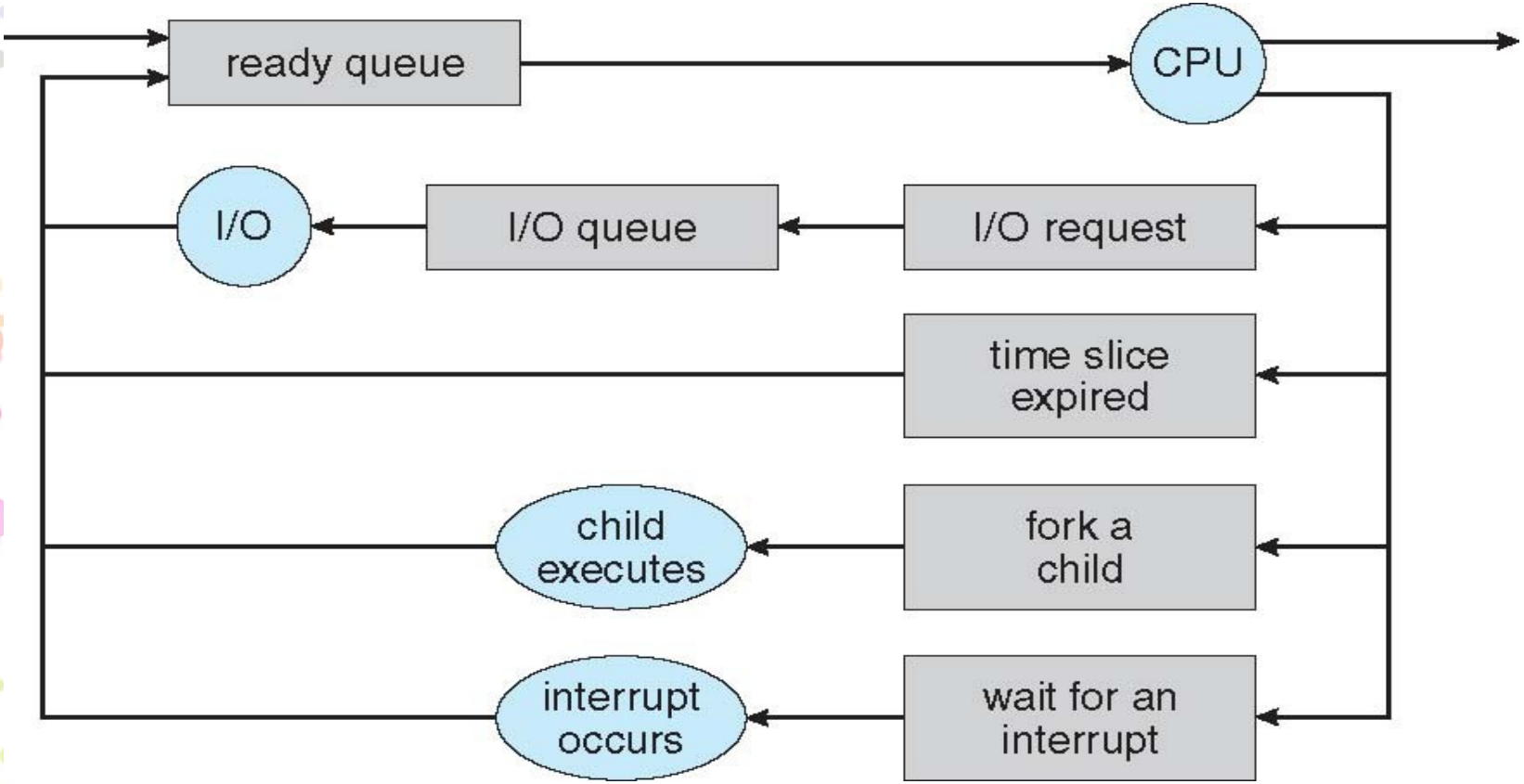
Scheduler

- The scheduler is part of the operating system, concerned with which process to run next if multiple run-able processes are in ready queue
- Scheduler is concerned with deciding on policy, not providing a mechanism
- It is important because it can have a big effect on resource utilization and overall performance of the system

Scheduling Algorithm

- In a multiprogramming computer several processes will compete for use of the processor
- At any time only one process will be running while other will wait for process to be free
- Scheduling algorithm is logic that determines the order in which processes should run when multiple processes are there
- The aim of process scheduling is to achieve objectives such as maximum CPU utilization and to improve its performance

Scheduling

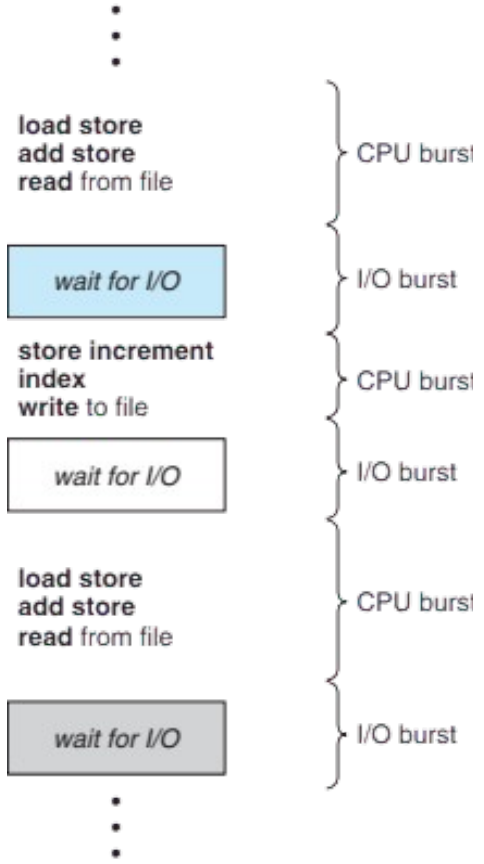

ready queue
CPU
I/O
I/O queue
I/O request
time slice expired
child executes
fork a child
interrupt occurs
wait for an interrupt

Process Scheduling

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait
- CPU burst followed by I/O burst
- CPU burst distribution is of main concern

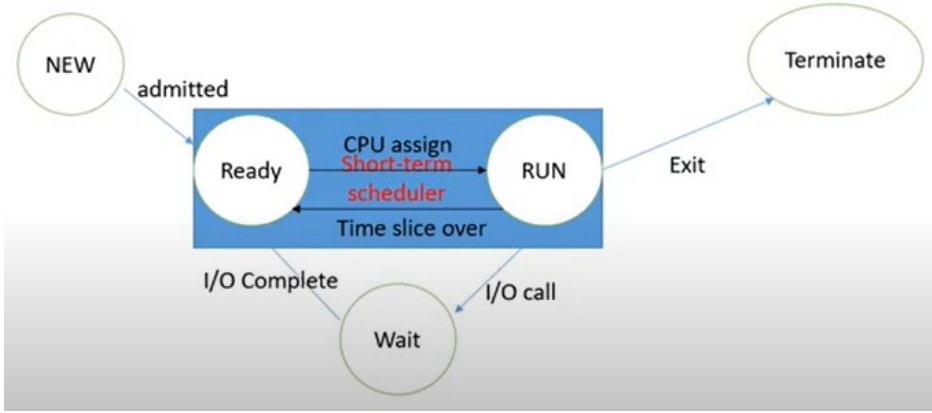
Objective

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time



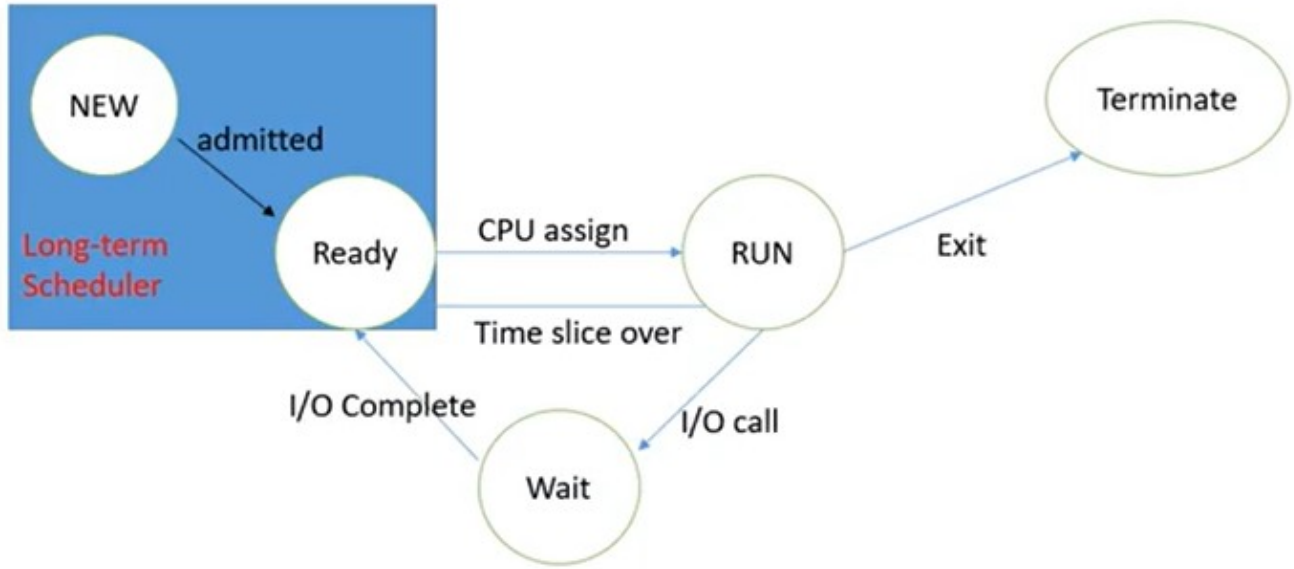
CPU Scheduler

- Short-term scheduler selects from among the processes in ready queue, and allocates the CPU to one of them work for **READY and RUN**
  - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
  - Switches from running to waiting state
  - Switches from running to ready state
  - Switches from waiting to ready
  - Terminates
- Scheduling under 1 and 4 is non-preemptive
- All other scheduling is preemptive
  - Consider access to shared data
  - Consider preemption while in kernel mode
  - Consider interrupts occurring during crucial OS activities



CPU Scheduler

Long-term scheduler bring the processes in ready queue, and work for **NEW and Ready**



Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- Dispatch latency
  - time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria/Objective

- CPU utilization
  - keep the CPU as busy as possible
- Throughput
  - # of processes that complete their execution per time unit
- Turnaround time
  - amount of time to execute a particular process
- Waiting time
  - amount of time a process has been waiting in the ready queue
- Response time
  - amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

- Class
  - Class of job, i.e. batch or on-line or real-time
- Resource requirements
  - E.g. expected run time, memory required, etc
- I/O or CPU bound
  - i.e. whether job is I/O bound or CPU bound
- Resources used to date
  - i.e. the amount of processor time already consumed
- Waiting time to date
  - i.e. the amount of time spent waiting for service so far
- Priority
  - Priority assigned to that process or job

Command line argument

In order to implement command line arguments, generally, 2 parameters are passed into the main function:

1. Number of command line arguments
2. The list of command line arguments

The basic syntax is

```
int main( int argc, char *argv[] )
{
.
.
// BODY OF THE MAIN FUNCTION
.
.
}
```

```
// The program name is cl.c
#include<stdio.h>
int main(int argc, char** argv)
{
printf("Hello World!\n\n");
int i;
printf("The number of arguments are: %d\n",argc);
printf("The arguments are:");
for ( i = 0; i < argc; i++)
{
printf("%s\n", argv[i]);
}
return 0;
}
```

argc: It refers to “argument count”. It is the first parameter that we use to store the number of command line arguments. It is important to note that the value of argc should be greater than or equal to 0.
agrv: It refers to “argument vector”. It is basically an array of character pointer which we use to list all the command line arguments.

https://www.onlinegdb.com/online_c_compiler#

Exec()

- Fork(): system call is used to create separate, duplicate process.
- Exec() the program specified in the parameter to exec() will replace the entire process including thread

- int execl(char const *path, char const *arg0, ...);
- int execle(char const *path, char const *arg0, ..., char const *envp[]);
- int execlp(char const *file, char const *arg0, ...);
- int execv(char const *path, char const *argv[]);
- int execve(char const *path, char const *argv[], char const *envp[]);
- int execvp(char const *file, char const *argv[]);
- int fexecve(int fd, char *const argv[], char *const envp[]);

Path: The argument specifies the path name of the file to execute as the new process image. Arguments beginning at arg0 are pointers to arguments to be passed to the new process image. The argv value is an array of pointers to arguments.

arg0: The first argument arg0 should be the name of the executable file. Usually it is the same value as the path argument. Some programs may incorrectly rely on this argument providing the location of the executable, but there is no guarantee of this nor is it standardized across platforms.

envp: Argument envp is an array of pointers to environment settings. The exec calls named ending with an e alter the environment for the new process image by passing a list of environment settings through the envp argument. This argument is an array of character pointers; each element (except for the final element) points to a null-terminated string defining an environment variable.

Exec()

e: It is an array of pointers that points to environment variables and is passed explicitly to the newly loaded process.
l: l is for the command line arguments passed a list to the function
p: p is the path environment variable which helps to find the file passed as an argument to be loaded into process.
v: v is for the command line arguments. These are passed as an array of pointers to the function.

- Execl() the calling process will be replaced with the new program where l represent list of parameters
- Code segment , data segment, heap and stack will be replaced. With the new program
- First argument is the path of the file.
- Second till end are rguments
- To Terminate pass NULL

- int excel(const char * path,) l is list of parameters.

Lab

- Run Command Line Argument
- Create a Process with EXEC()



Thank you